slanglabs

Makers of Conva.AI — The world's first AI Assistant as a Service platform, enabling businesses to easily create, deploy, integrate, observe, eval and maintain awesome AI experiences inside their apps

Follow publication

How to Use an External Docker Registry and Key Vault Secrets with Azure Container Apps

Anmol Prakash · Follow · 10 min read · Sep 30, 2024

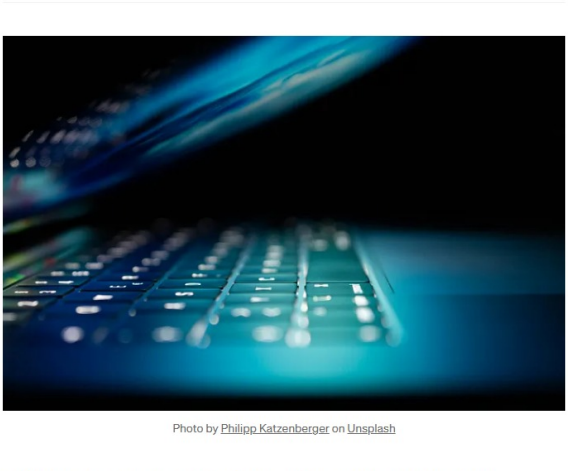


Photo by Philipp Katzenberger on Unsplash

In Slang Labs, we have a very unique and innovative product called CONVA.AI which has multiple micro services written in Python language. CONVA.AI is the world's first multi cloud AI assistant as a service platform and we wanted to use a cloud agnostic docker registry which can be used to store and pull docker images from any of the cloud platforms and that is where Gitlab Registry turned out to be a good choice for storing docker images.

Our micro services are hosted as Azure Container Apps and it is also a fine choice for deploying micro-services on a scalable, serverless platform.

If you also want to choose Azure Container apps and use it along with GitLab Registry to store your Docker images and want to keep your secrets safe with Azure Key Vault, you're in the right place.

In this blog, we'll explore how to set up and deploy a python-based microservice to Azure Container Apps from a GitLab Registry, using secrets stored in Azure Key Vault.

Prerequisites

Before you begin, ensure you have the following:

1. An Azure account
2. Azure CLI installed locally
3. Access to your GitLab Registry
4. Access to Azure Key Vault

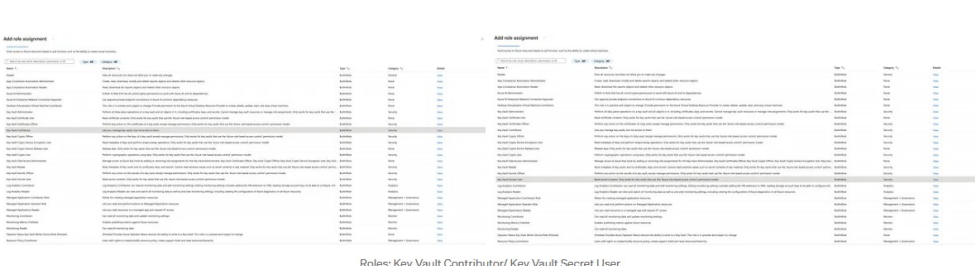
Step 1: Store Docker Credentials in Key Vault

First, store your GitLab Docker Registry credentials inside Azure Key Vault. This step ensures that any authentication credentials are kept secure and can be accessed by your container app without hardcoding them.

Prerequisites:

Azure Subscription administrator should assign your Microsoft Entra ID account the following 2 roles:

- Key Vault Contributor
- Key Vault Secrets User



Roles: Key Vault Contributor/ Key Vault Secret User

Steps:

- Login to the Azure Portal (https://portal.azure.com/).
- Navigate to "Key Vaults" and click "Create."
- Fill in the necessary details (Resource Group, Key Vault Name, Location, etc.).
- Click "Review + Create" and then "Create."
- After your Key Vault is created, navigate to it.
- Click on "Secrets" in the left-hand menu and then "Generate/Import."
- In the "Create a secret" section, fill in the Name and Value fields:
  - Name: "gitlab-registry-password`
  - Value: Your GitLab Password

Step 2: Prepare your Python-based Microservice

Let's get started and first create a simple Hello World Application. I am going to use the FastAPI framework to build a lightweight Hello World API Service.

- Open a blank new folder where you want to keep the code related to Hello World API Service.
- You can open the folder using Visual Studio Code or PyCharm.
- Add a new file *main.py* inside your project folder
- Add the following code under *main.py*:

```
import os
from fastapi import FastAPI

app = FastAPI()
@app.get("/")
def hello_world():
 return {"text": "Hello World!, I am running Fast API"}
```

- Under the root folder of your project, add a file named requirements.txt . This file will store all the dependencies which are required for the project. Add the following dependencies under this file:

```
fastapi==0.111.0
uvicorn==0.29.0
```

- Next open the Terminal on your vs code and enable python virtual environment by executing the following commands.:

```
➜ hello-world-api python -m venv venv
➜ hello-world-api source venv/bin/activate
(venv) ➜ hello-world-api
```

- Install dependencies:

```
(venv) ➜ hello-world-api pip install -r requirements.txt
```

- Run the server locally and check if the API is working :

```
(venv) ➜ hello-world-api uvicorn main:app - port 8080
INFO: Started server process [33792]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: 127.0.0.1:59140 - "GET / HTTP/1.1" 200 OK
```

- You should be able to access the APi on your browser at http://localhost:8080. It should return the following:

```
{"text":"Hello World!, I am running Fast API"}
```

When everything is working locally, we can go onto the step for creating a Dockerfile.

Step 3: Dockerfile for the Python based API Service

Once you have created the python API service, now it's time to containerize our API service. We will define a Dockerfile which will take care of containerizing our API service. Add a file named *Dockerfile* under the root of your project directory.

Add the following configuration under it:

```
FROM --platform=linux/amd64 python:3.12
WORKDIR /app
COPY ./requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", " - host", "0.0.0.0", " - port", "80"]
```

Now build a docker image, tag it and push it to your gitlab docker registry.

```
(venv) ➜ hello-world-api docker build -t registry.gitlab.com/<gitlab-org-name>/
(venv) ➜ hello-world-api docker push registry.gitlab.com/<gitlab-org-name>/<git
```

Time to now create a container app in azure. But wait, if you have never created a container app before, you will have to create a container app environment first.

Step 4: Create a Container App Environment

You can directly go to "Step 5" if you already have a container app environment else follow the steps given below:

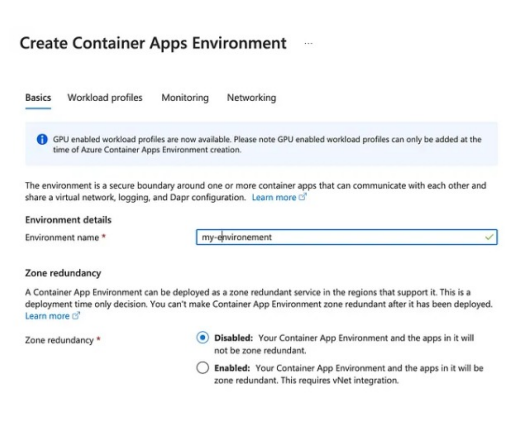
Steps:

1. Navigate to the Azure Portal.

2. Search for "Container Apps" and click "Create".

3. In Container Apps Environment select "Create New"

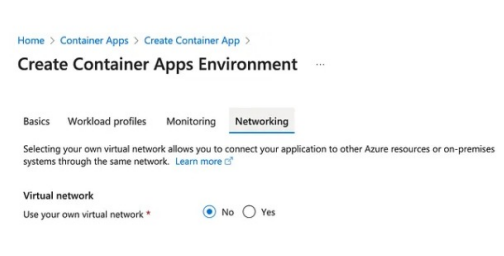
4. Choose a name for your environment.



5. Choose Workload Profiles. By default, a consumption profile with 4 cpu and 8 GB size will be automatically included. To keep this blog simple, let's continue with the default consumption plan.

6. In the monitoring tab, you can choose the service which you want to use as a destination to store container logs. Both Azure Log analytics and Azure Monitor are good options. Let's go with Azure Log Analytics for the sake of this blog.

7. Under the "Networking" tab, you can either choose your own VPC and subnet for the container app environment or choose no virtual network . If you choose your own VPC then as a prerequisite, you will have to create a VPC and subnet which will host all the container app resources. Selecting your own VPC and subnet allows you to connect your application to other Azure resources in the same network



8. Click Create to provision your container app environment.

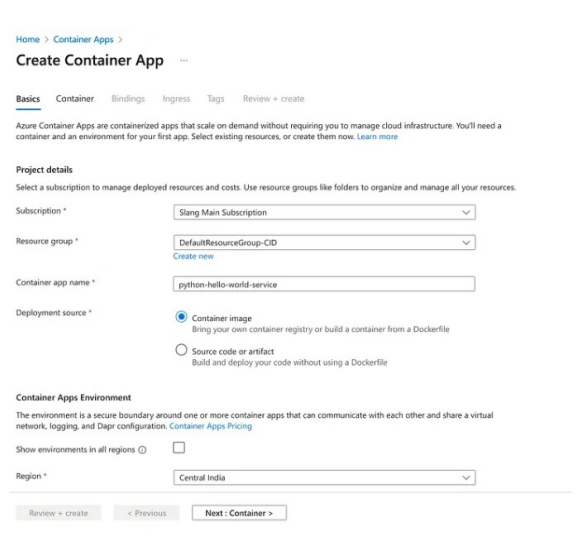
Once the Container app environment is created, let's continue with the container app(micro-service) creation now.

Note: A container app environment gets created only when the first container app is deployed in that environment. Thus the process of creating a container app env is dependent on creating your first container app

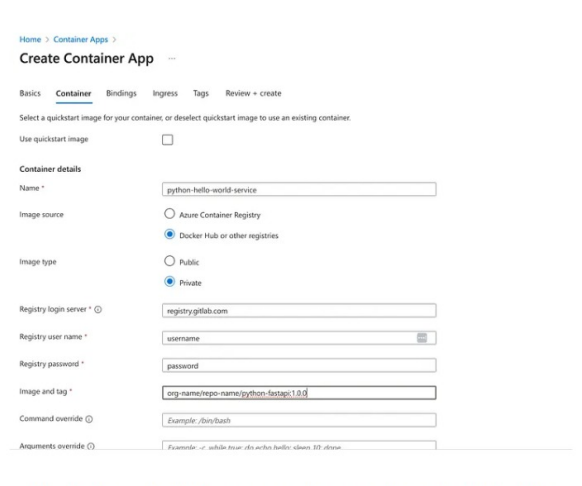
Step 5: Create a Container App and Deploy Your Python Service

Now, let's deploy the Docker image of your Python-based microservice from the GitLab Registry into a container app

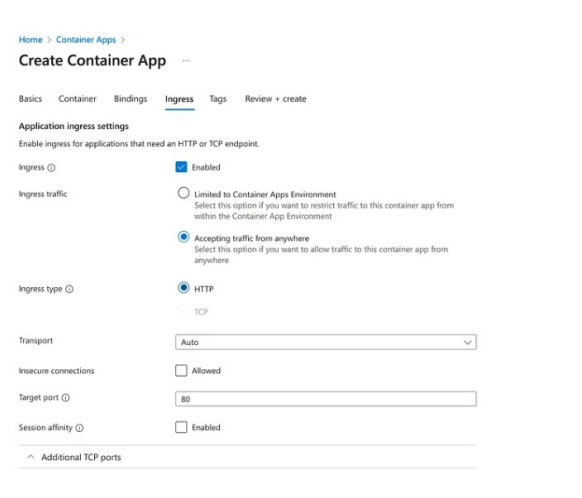
- Navigate to the Azure Portal.
- Search for "Container Apps" and click "Create".
- Provide the name, resource group, and environment details.



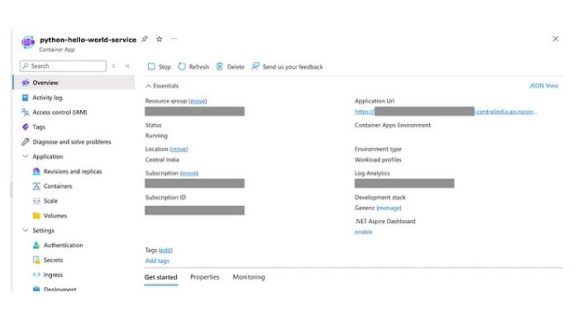
- For "Image source," select "Docker Hub or other registries".
- Enter the GitLab Registry URL and image name.



- Provide the credentials (username and password) to authenticate with gitlab registry. When you enter the credentials for the registry from the Azure portal page to create the container app, the authentication details will be stored as container app secret. We will shortly see how to make use of the Azure key vault secret with Azure container app.
- Under the Ingress tab, enable the ingress and select "Accept traffic from anywhere". The target port will be the same port where our python service will run within the container. In our case the uvicorn server will startup at port 80. So set the target port also to 80.



- Go to Review and Create and Click "Create" once all the validation is passed
- Once the container app is successfully provisioned, the application URL can be found under the overview section of the container app. This url can be used to access the python based FastAPI service which we just deployed in the Azure container app.



. . .

Using Azure CLI (Alternate approach for creating a container app)

Alternatively, if you prefer to use CLI, you can create a YAML file which helps in maintaining the configuration of your container in the form of a template. You can also commit this template into your git repository which can be later used for updating the same container app or creating a copy of a similar type of a container app.

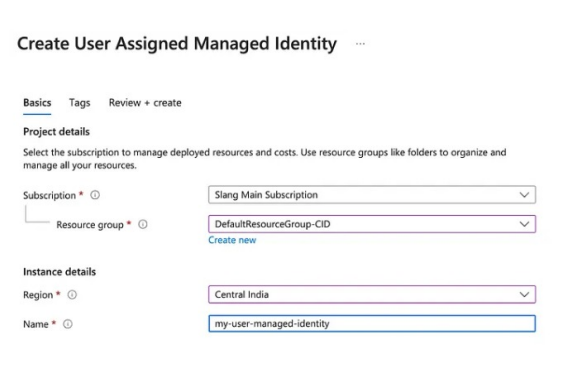
Prerequisites:

1. Install Azure CLI
2. Create a User Assigned Managed Identity and give it access to the azure key vault where the secrets are stored. This managed identity will be used by the container app to fetch the secrets from the azure key vault.

Create a User Assigned Managed Identity

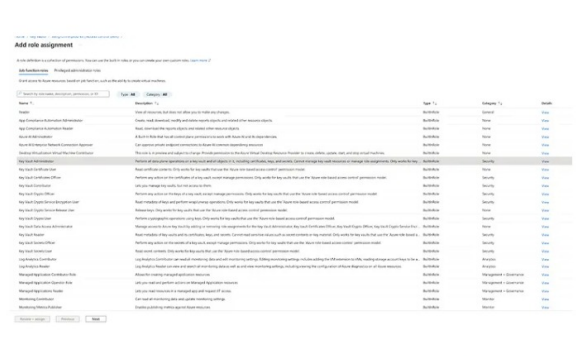
To create a user defined managed identity and provide it access to the azure key vault, follow the steps below:

- Login to Azure portal.
- Search for "Managed Identities" in the search bar.
- Create a managed identity and provide the necessary details such as name, resource group, location etc.

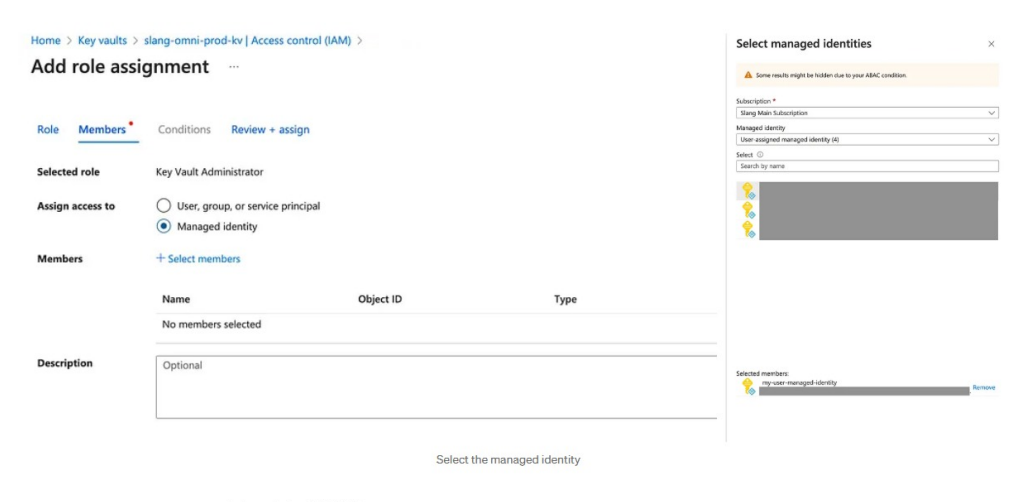


- Once the Managed Identity is created, note down the clientID and principal id from the overview section of the just created managed Identity. This will be required for configuring the YAML template of the container app later.

- Next, go to the azure key vault and select the key vault under which you want to give access to the user defined managed identity which you just created.
- Go to Access Control(IAM) and select Key Administrator as the role to assign.



- On the Members page, select Managed Identity and search for the name of your managed identity



Select the managed identity

Template (YAML)

```
name: python-hello-world-service
location: centralindia
resourceGroup: <resource-group-name>
type: Microsoft.App/containerApps
identity:
  type: UserAssigned
  userAssignedIdentities:
    ? "/subscriptions/<subscription-id>/resourcegroups/<resource-group-name>/pro
    : {"principalId": "<principalId>", "clientId": "<clientId>"}
properties:
  environmentId: /subscriptions/<subscription-id>/resourceGroups/<resource-group
  workloadProfileName: Consumption
  configuration:
    activeRevisionsMode: Single
    ingress:
      external: true
      allowInsecure: false
      clientCertificateMode: ignore
      targetPort: 80
      traffic:
      - latestRevision: true
        weight: 100
      transport: Auto
    registries:
    - identity: ''
      passwordSecretRef: gitlab-password
      server: registry.gitlab.com
      username: <gitlab-username>
    secrets:
    - identity: '/subscriptions/<subscription-id>/resourcegroups/<resource-group
      keyVaultUrl: https://<key-vault-name>.vault.azure.net/secrets/gitlab-passw
      name: gitlab-password

  template:
    revisionSuffix: ''
    containers:
    - image: 'registry.gitlab.com/<gitlab-org-name>/<gitlab-repo-name>/python-fa
      name: python-hello-world-service
      resources:
        cpu: 0.5
        memory: 1Gi
    scale:
      maxReplicas: 10
      minReplicas: 1
```

Some parameters which needs to be replaced as per your Azure account and environment in the above template are:

1. **<resource-group-name>** — Name of the resource group under which you want to deploy the container app
2. **<subscription-id>** — Your azure account's active subscription id.
3. **<managed-identity-name>** — Name of the managed identity which we created in the section "Create User Assigned Managed Identity"
4. **<principalId>** — PrincipalId of the managed identity
5. **<clientId>** — ClientID of the managed identity
6. **<container-app-evn-name>** — Name of the container app environment
7. **<gitlab-username>** — Username for the gitlab account
8. **<gitlab-org-name>** — Org name of the gitlab account.
9. **<gitlab-repo-name>** — Repo name under the gitlab under which container registry is being used

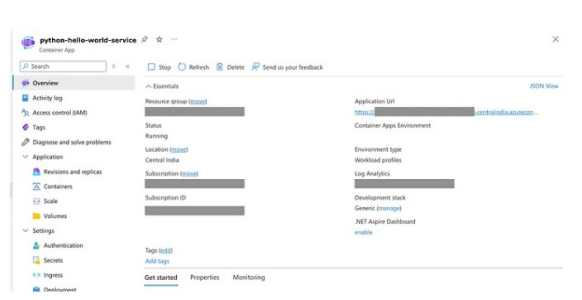
Deployment using CLI

In the bash shell, sign in to Azure using **az login.** Make sure the account which you use to sign in to Azure has either contributor or owner level access within the azure subscription.

```
(venv) ➜ hello-world-api: az login
(venv) ➜ hello-world-api: az extension add --name containerapp --upgrade
(venv) ➜ hello-world-api: az containerapp create -n python-hello-world-service
```

And there you have it!. The above command will create a container app with the name "python-hello-world-service".

Once the service is up and runnign, It will be assigned a public facing endpoint URL by Azure. You will find the endpoint URL under the **Overview** section of the container app by the label name: **Application Url**.



Wrapping Up

Hope this blog has been helpful to those who are deploying a Python-based micro service from GitLab Registry into Azure Container Apps, ensuring secrets safety using Azure Key Vault.

Slang Labs with their product CONVA.AI stands as a testimony to how powerful and flexible this deployment model can be. Utilizing Azure Container Apps with external Docker registries and secure key storage can significantly enhance the scalability and security of your applications.

Now, go forth and deploy your own micro services with confidence, knowing your secrets and configurations are well guarded!

Happy Deploying!

Azure Container Apps · Docker · Gitlab Registry · Azure Key Vault · Azure Cli

Published in slanglabs

105 followers · Last published Oct 30, 2024

Makers of Conva.AI — The world's first AI Assistant as a Service platform, enabling businesses to easily create, deploy, integrate, observe, eval and maintain awesome AI experiences inside their apps

Follow

Written by Anmol Prakash

2 followers · 3 following

Follow